

Quick Update:

So let's do some agentic programming I have the the PI agent installed, so here we are these are some old tests that I created some time ago they should more or less work. but let's see what this beast can do I want him to test my website. let's try with localhost so he doesn't fuck things up. let's create a test plan have a look at this web page and create a test okay that's that's very very simple can we make this bigger is much nicer. let's do a simple come on let's do a simple sanity test, after you had a look around create a test plan called Jeff Meridian localhost test suite or something. okay now I'm gonna hit enter let's see what happens he's working on it that's fast there's hardly I'm hardly able to read that.. is some scary shit... oh he's fast... now.... okay at least he takes it's time.... now working on the test plan.... if he's... if he's.. so fast.... that scares me... oh all right what now Jesus Christ the inference speed is insane.... I'm speechless for a forehandling you can run the test Jesus Christ let's have a look what he's doing check page a title visible your allies change okay interesting interest let's run them all in failing failing this thing it's fairly lame it's a very some stupid shit .. lets download some browsers except we don't whatever we don't happens now we have browsers expect curious if you runston headless no he's not he's open when that's my bad I should have restrained him we open a gazillion of window... maybe my computer can handle it... can we handle it.... oh yeah magic.... I'm a bit pissed... if it's

okay... at least it's failing in..... this just kind of makes me dizzy.... just looking at it... and it's hard to imagine... I hope he's going to close all the browsers..... that he's opening... this is fucking up with my recording.... because the frame rates are dropping..... panic..... should I stop the tests..... this is crazy..... I'm still in shock..... Jesus Christ..... all right we are so lucky because I just checked and we don't have a gazillion of chrome instances open so that's a good sign I don't want to debug those I think that's why we have the healer this is already amazing let's just run one failing text to display the language we see if it was a performance nope he has a problem actually a great example to see what the what the AI coding agent can do..... gonna use our new agent... please go ahead and no please fix this failing test.... heal this failing test... let's see what he is doing.... cool Wow, alright..., so that's what's going on.....

Introduction to Pi

Pi is a minimal terminal coding harness that puts you in control. Unlike other coding agents, Pi doesn't force you into a specific workflow. Instead, you adapt Pi to match how you work — without forking or modifying Pi internals.

```
tools?: Tool[];
commands?: Command[];
middleware?: Middleware[];
activate: (context: ExtensionContext)
=> Promise<void>;
deactivate: () => Promise<void>;
}
```

Additional Resources

- [GitHub Repository](#)
- [Discord Community](#)
- [Blog Post](#)
- [Examples](#)
- [GitHub Issues](#)

Last updated: 2026-04-27

What Makes Pi Different

Pi ships with powerful defaults but intentionally skips features like sub-agents and plan mode. Instead, you can:

- Ask Pi to build exactly what you need
- Install third-party Pi packages that match your workflow
- Create custom extensions, skills, and prompt templates
- Share your customizations via npm or git

Key Features

Four Flexible Modes

1. **Interactive:** Full terminal UI with session management
2. **Print/JSON:** One-shot responses for scripting
3. **RPC:** Process integration over stdin/stdout
4. **SDK:** Embed Pi in your own applications

Extensibility First

- **Extensions:** TypeScript modules for custom tools, commands, and UI

- **Skills:** On-demand capabilities following Agent Skills standard
- **Prompt Templates:** Reusable prompts with variable substitution
- **Themes:** Customizable colors and styles with hot-reload
- **Pi Packages:** Bundle and share via npm or git

Multi-Provider Support

Pi supports 20+ LLM providers through unified APIs:

Subscriptions (OAuth):

- Anthropic Claude Pro/Max
- OpenAI ChatGPT Plus/Pro (Codex)
- GitHub Copilot
- Google Gemini CLI
- Google Antigravity

API Keys:

- Anthropic, OpenAI, Azure OpenAI
- Google Gemini, Vertex AI, Amazon Bedrock
- Mistral, Groq, Cerebras, xAI
- OpenRouter, Vercel AI Gateway, and more

Tool Interface

Tools follow a standard interface:

```
interface Tool {
    name: string;
    description: string;
    parameters: z.ZodSchema;
    execute: (params: any, context:
ToolContext) => Promise<ToolResult>;
}

interface ToolContext {
    cwd: string;
    sessionId: string;
    signal: AbortSignal;
}

interface ToolResult {
    output?: string;
    error?: string;
    files?: Array<{ path: string; content:
string }>;
}
}
```

Extension Interface

Extensions follow a standard interface:

```
interface Extension {
    name: string;
    description?: string;
}
```

Agent Core API

Agent class

Main agent class for running conversations.

Constructor options:

- `provider`: Provider instance
- `tools`: Array of tools
- `systemPrompt`: System prompt (optional)
- `maxContextTokens`: Maximum context tokens (optional)

Methods:

- `complete(options)`: Run a completion
- `addTool(tool)`: Add a tool
- `removeTool(name)`: Remove a tool
- `getHistory()`: Get conversation history
- `clearHistory()`: Clear conversation history

Example:

```
const agent = new Agent({
  provider,
  tools: [read, write, edit, bash]
});

const response = await agent.complete({
  messages: [{ role: 'user', content:
'Hello' }]
});
```

- Any OpenAI-compatible API (Ollama, vLLM, LM Studio)

Session Management

- Sessions stored as JSONL with tree structure
- In-place branching without creating new files
- `/tree` command to navigate and switch branches
- Manual and automatic context compaction
- Export to HTML or share as GitHub gist

Built-in Tools

Pi gives models access to your codebase through tools:

- `read` - Read files with offset/limit support
- `write` - Create new files
- `edit` - Make precise edits to existing files
- `bash` - Execute shell commands
- `grep`, `find`, `ls` - Optional file search tools

Extensions can replace or augment these with custom implementations.

Philosophy

Note: Pi is aggressively extensible so it doesn't have to dictate your workflow. Features that other tools bake in can be built with extensions, skills, or installed from third-party Pi packages.

- **No MCP** — Build CLI tools with READMEs (see Skills), or build an extension that adds MCP support.

- **No sub-agents** — Spawn Pi instances via tmux, or build your own with extensions, or install a package.
- **No permission popups** — Run in a container, or build your own confirmation flow with extensions.

- **No plan mode** — Write plans to files, or build it with extensions, or install a package.
- **No built-in to-dos** — They confuse models. Use a TODO.md file, or build your own with extensions.

- **No background bash** — Use tmux for full observability and direct interaction.

Read the [blog post](#) for the full rationale.

Pi is distributed as an npm package and requires Node.js 20 or later.

Quick Start Install Pi

Parameters:

- options.provider: Provider instance
- options.messages: Array of message objects
- options.tools: Array of available tools (optional)
- options.maxTokens: Maximum tokens (optional)
- options.temperature: Temperature (optional)

Returns: Async iterable of stream events

Example:

```
const stream = await streamAndComplete({
  provider,
  messages: [{ role: 'user', content:
    'Hello' }],
  tools: [read, write, bash]
});

for await (const event of stream) {
  if (event.type === 'text') {
    console.log(event.text);
  } else if (event.type === 'toolCall') {
    {
      console.log('Tool call:',
        event.toolCall);
    }
  }
}
```

API Reference

AI Package API

createProvider(options)

Create an LLM provider instance.

Parameters:

- `options.provider` (string): Provider name (anthropic, openai, etc.)
- `options.apiKey` (string): API key
- `options.model` (string): Model name
- `options.baseUrl` (string): Custom base URL (optional)

Returns: Provider instance

Example:

```
const provider = createProvider({
  provider: 'anthropic',
  apiKey: 'sk-ant-...',
  model: 'claude-sonnet-4-5-20250929'
});
```

streamAndComplete(options)

Stream a completion response.

Step 1: Install Pi globally

```
npm install -g @mariozechner/pi-coding-agent
```

This makes the `pi` command available system-wide.

Step 2: Set up authentication

Choose one of two authentication methods:

API Key (Simple):

Set an environment variable for your preferred provider:

```
export ANTHROPIC_API_KEY=sk-ant-...
```

***Tip:** Add this to your `~/ .bashrc` or `~/ .zshrc` to make it permanent.*

Supported providers:

- ANTHROPIC_API_KEY - Anthropic
- OPENAI_API_KEY - OpenAI
- GEMINI_API_KEY - Google Gemini
- GROQ_API_KEY - Groq
- MISTRAL_API_KEY - Mistral
- And [many more](#)

Subscription (OAuth):

For Pro/Plus subscribers (Claude Pro, ChatGPT Plus, etc.), Pi supports OAuth authentication.

Step 3: Launch Pi

pi
You'll see the interactive terminal interface with an editor at the bottom.

Step 4: Start coding

Type your request and press Enter:
List all TypeScript files in this directory

Pi will use the built-in tools (read, write, edit, bash) to fulfill your request.

Your First Session

Let's walk through a typical Pi session:

```
# Start Pi in your project directory
cd ~/my-project
pi
# Pi loads and shows the interface
# Type your first message:
"Create a simple HTTP server in Node.js
that responds with 'Hello World'"
```

```
npm install @mariozeczner/pi-web-ui
```

@mariozeczner/pi-mom

Slack bot integration for Pi.

Features:

- Slack integration
- Channel-based conversations
- File sharing
- Command support

Installation:

```
npm install @mariozeczner/pi-mom
```

@mariozeczner/pi-pods

CLI for managing VLLM deployments.

Features:

- VLLM deployment management
- Model hosting
- API endpoint management
- Resource monitoring

Installation:

```
npm install @mariozeczner/pi-pods
```

- Error recovery

Installation:

```
npm install @mariozechner/pi-agent-core
```

@mariozechner/pi-tui

Terminal UI library used by Pi.

Features:

- Reactive UI components
- Syntax highlighting
- Auto-completion
- Keyboard navigation

Installation:

```
npm install @mariozechner/pi-tui
```

@mariozechner/pi-web-ui

Web components for building AI chat interfaces.

Features:

- Chat UI components
- Message rendering
- Code highlighting
- Theme support

Installation:

```
# Pi will:
# 1. Create a new file (server.js)
# 2. Write the code
# 3. Show you what it did
```

```
# Continue the conversation:
"Add a /status endpoint that returns
JSON"
```

```
# Pi edits the existing file and shows
the changes
```

***Note:** Sessions auto-save to ~/.pi/agent/sessions/ organized by working directory. Use pi -C to continue your most recent session.*

Essential Commands

Type / in the editor to see all commands. Here are the most useful:

Command Description /model Switch between LLM models /settings Adjust thinking level, theme, and more /tree Navigate session history and branches /copy Copy last assistant message to clipboard /hotkeys Show all keyboard shortcuts /quit Exit Pi

Essential Keyboard Shortcuts

Key Action `Ctrl+L` Open model selector `Ctrl+P / Shift+Ctrl+P` Cycle through models `Shift+Tab` Cycle thinking level (off → minimal → low → medium → high → xhigh) `Escape` Cancel current operation `Escape` twice Open session tree view `Ctrl+C` Clear editor (press twice to quit) `Ctrl+O` Collapse/expand tool output `@` Fuzzy-search and reference files

Tip: Use Shift+Enter (or Ctrl+Enter on Windows Terminal) for multi-line input.

File References

Include files in your messages using the `@` symbol:

`@src/server.ts` Review this file for security issues

Type `@` and start typing to get fuzzy-search suggestions. You can reference multiple files:

`@package.json @README.md` Update the dependencies and document the changes

- Multi-model support

Installation:

```
npm install -g @mariozechner/pi-coding-agent
```

@mariozechner/pi-ai

Unified API for multiple LLM providers.

Features:

- 20+ provider support
- Unified interface
- Streaming support
- Token counting
- Cost tracking

Installation:

```
npm install @mariozechner/pi-ai
```

@mariozechner/pi-agent-core

Core agent runtime with tool calling capabilities.

Features:

- Tool calling framework
- Message management
- Context handling

JSON Mode

Get structured JSON output:

```
pi --json "List all TypeScript files"
```

Packages Overview

Pi is built as a monorepo with multiple packages:

Package Description [@mariozechner/pi-coding-agent](#)
Interactive coding agent CLI [@mariozechner/pi-ai](#) Unified
multi-provider LLM API [@mariozechner/pi-agent-core](#)
Agent runtime with tool calling [@mariozechner/pi-tui](#)
Terminal UI library [@mariozechner/pi-web-ui](#) Web
components for AI chat [@mariozechner/pi-mom](#) Slack bot
integration [@mariozechner/pi-pods](#) CLI for managing
vLLM deployments

@mariozechner/pi-coding-agent

The main CLI package for interactive coding assistance.

Features:

- Interactive terminal UI
- Session management
- Tool execution
- Extension support

Running Bash Commands

Prefix commands with ! to run them and send output to the LLM:

```
!ls -la src/
```

Use !! to run without sending output:

```
!!npm install lodash
```

Session Management

- **Continue Previous Session:** Use `pi -c` to continue your most recent session
- **Browse All Sessions:** Sessions are stored in `~/.pi/agent/sessions/`
- **Branch from History:** Use `/tree` to navigate and branch from any point
- **Export or Share:** Export sessions to HTML or share as GitHub gist

Switching Models

Pi supports 20+ LLM providers. Switch models at any time:

1. **Open model selector:** Press `Ctrl+L` or type `/model`
2. **Choose your model:** Use arrow keys and search to find your model. The list shows:

- Model name and provider
- Context window size
- Pricing (input/output tokens)
- Whether it supports vision and reasoning

1. **Continue conversation:** Your session continues with the new model. Pi handles cross-provider compatibility automatically.

Note: Use `pi -models "claude-*, gpt-4o"` to limit `Ctrl+P` cycling to specific model patterns.

Troubleshooting

Pi command not found

Ensure Pi was installed globally and your npm bin directory is in PATH.

```
import { createProvider } from '@mariozechner/pi-ai';
```

```
const provider = createProvider({
  provider: 'anthropic',
  apiKey: process.env.ANTHROPIC_API_KEY
});
```

```
const agent = new Agent({
  provider,
  tools: [read, write, edit, bash]
});
```

```
const response = await agent.complete({
  messages: [{ role: 'user', content:
    'Hello' }]
});
```

RPC Mode

Run Pi in RPC mode for process integration:

```
pi --rpc
```

Communicate over stdin/stdout with JSON messages.

Print Mode

Get one-shot responses for scripting:

```
pi --print "List all TypeScript files"
```

```

    "thinkingLevel": "medium",
    "defaultModel": "claude-
sonnet-4-5-20250929",
    "models": ["claude-*", "gpt-4o"],
    "extensions": ["my-extension"],
    "skills": ["my-skill"],
    "tools": {
      "read": true,
      "write": true,
      "edit": true,
      "bash": true
    },
    "sessionDir": "~/.pi/agent/sessions",
    "maxContextTokens": 100000,
    "compactStrategy": "auto"
  }
}

```

Programmatic Usage

SDK

Pi can be embedded in applications using the SDK.

Installation:

```
npm install @mariozechner/pi-agent-core
@mariozechner/pi-ai
```

Basic usage:

```
import { Agent } from '@mariozechner/pi-
agent-core';
```

Authentication errors

Verify your API key environment variables are set correctly.

Windows: bash not found

Install Git Bash or WSL2 for bash support.

Node.js version errors

Pi requires Node.js 20 or later. Update with `nvm install 20`.

***Tip:** For more help, join the [Discord community](#) or check the [GitHub issues](#).*

Installation

System Requirements

- **Node.js:** Version 20 or later

- **npm:** Latest version recommended
- **Operating Systems:** macOS, Linux, Windows (with WSL2 or Git Bash)

Global Installation

```
npm install -g @mariozechner/pi-coding-agent
```

Verification

```
pi --version
```

API Key Providers

Set environment variables for your preferred providers:

```
# Anthropic
export ANTHROPIC_API_KEY=sk-ant-...

# OpenAI
export OPENAI_API_KEY=sk-...

# Google Gemini
export GEMINI_API_KEY=...
```

14

Prompt Templates

Create reusable prompts with variable substitution.
Example template:

Review the following code for best practices:

Focus on:

- Security issues
- Performance
- Readability

Using templates:

```
/template review-code  
language=typescript code=@src/file.ts
```

Extensions

See the [Extension System](#) section for details.

Configuration

Full configuration options in ~/.pi/config.json:

```
{  
  "theme": "dark",  
}
```

27

- high-contrast

Custom themes:

Create ~/.pi/themes/my-theme.json:

```
{
  "name": "my-theme",
  "colors": {
    "background": "#000000",
    "foreground": "#ffffff",
    "primary": "#00ff00",
    "error": "#ff0000"
  }
}
```

Skills

Skills are on-demand capabilities following the Agent Skills standard.

Using skills:

```
/skill install <skill-name>
/skill list
/skill enable <skill-name>
```

Creating skills:

Skills are CLI tools with a specific interface. See the Skills documentation for details.

```
# Groq
export GROQ_API_KEY=...
```

```
# Mistral
export MISTRAL_API_KEY=...
```

```
# Azure OpenAI
export AZURE_OPENAI_API_KEY=...
export AZURE_OPENAI_ENDPOINT=...
```

```
# Amazon Bedrock
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_REGION=...
```

```
# And many more...
```

OAuth Authentication

For Pro/Plus subscribers, Pi supports OAuth authentication for:

- Anthropic Claude Pro/Max
- OpenAI ChatGPT Plus/Pro
- GitHub Copilot
- Google Gemini CLI

Run `pi auth` to start the OAuth flow.

Custom Configuration

Create a configuration file at `~/ .pi/config.json`:

```
{
  "theme": "dark",
  "thinkingLevel": "medium",
  "defaultModel": "claude-sonnet-4-5-20250929",
  "extensions": [],
  "skills": []
}
```

Core Concepts

Architecture

- Pi is built as a modular system with the following components:
1. **CLI Interface:** Terminal-based user interface
 2. **Agent Core:** Runtime for tool calling and conversation management
 3. **AI Layer:** Unified API for multiple LLM providers
 4. **Tool System:** File operations, bash execution, and search
 5. **Extension System:** Custom tools, commands, and UI components

16

Thinking Levels

- Adjust the thinking level with Shift+Tab:
- **off:** No thinking tokens
 - **minimal:** Minimal thinking
 - **low:** Low thinking
 - **medium:** Medium thinking (default)
 - **high:** High thinking
 - **high:** Maximum thinking

Cost Management

- View token usage in the status bar
- Set spending limits in config
- Use cheaper models for simple tasks

Customization

Themes

- Pi supports custom themes with hot-reload.
- Built-in themes:**
- dark (default)
 - light

25

- OpenAI
- Azure OpenAI
- Google Gemini
- Google Vertex AI
- Amazon Bedrock
- Mistral
- Groq
- Cerebras
- xAI (Grok)
- OpenRouter
- Vercel AI Gateway
- Any OpenAI-compatible API

Model Selection

Use `Ctrl+L` or `/model` to open the model selector. The list shows:

- Model name and provider
- Context window size
- Pricing (input/output per million tokens)
- Vision support
- Reasoning capabilities

6. **Session Store:** JSONL-based session storage with branching

Tool System

Pi provides models with tools to interact with your codebase:

Built-in Tools

- **read:** Read file contents with offset/limit support
- **write:** Create new files
- **edit:** Make precise edits using text replacement
- **bash:** Execute shell commands
- **grep:** Search files with regex
- **find:** Find files by name/pattern
- **ls:** List directory contents

Tool Configuration

Tools can be enabled/disabled via configuration:

```
{
  "tools": {
    "read": true,
    "write": true,
    "edit": true,
    "bash": true,
```

```
    "grep": false,
    "find": false,
    "ls": false
  }
}
```

Custom Tools

Extensions can provide custom tool implementations:

```
export const tool: Tool = {
  name: 'customTool',
  description: 'Does something custom',
  parameters: z.object({
    param: z.string()
  }),
  execute: async (params, context) => {
    // Implementation
  }
};
```

Extensions are TypeScript modules that extend Pi's functionality.

Extension System

Message Queue

- Messages are queued and processed sequentially
- Cancel current operation with `Escape`
- View queue status in the status bar

Tool Output

- Tool output is displayed below the conversation
- Collapse/expand with `Ctrl+O`
- Click to view full output

Providers & Models

Supported Providers

- Subscription-based (OAuth): Anthropic Claude Pro/Max
- Anthropic Claude Pro/Max
- OpenAI ChatGPT Plus/Pro
- GitHub Copilot
- Google Gemini CLI
- Google Antigravity
- API Key-based: Anthropic

Guides

Interactive Mode

Interface Overview

The interactive mode provides:

- Conversation history at the top
- Editor at the bottom
- Status bar with current model and thinking level
- Tool output display

Editor Features

- Multi-line input with `Shift+Enter`
- File references with `@`
- Command palette with `/`
- Syntax highlighting
- Auto-completion

Creating an Extension

1. Create a directory in `~/ .pi/extensions/`
2. Add an `index.ts` file with your extension code
3. Enable the extension in config

Extension Capabilities

Extensions can:

- Add custom tools
- Add custom commands
- Modify the UI
- Replace built-in tools
- Add middleware

Example Extension

```
import { Extension, Tool } from
'@mariozechner/pi-coding-agent';

export const extension: Extension = {
  name: 'my-extension',
  tools: [
    {
      name: 'hello',
      description: 'Say hello',
      parameters: z.object({
        name: z.string()
      })
    }
  ]
}
```

Session Management

```
    },
    execute: async (params) => {
        return `Hello, ${params.name}!`;
    }
}
];
};
```

Session Storage

Sessions are stored as JSONL files in ~/.pi/agent/sessions/, organized by working directory.

Session Structure

```
{
  "id": "session-id",
  "createdAt": "2024-01-01T00:00:00Z",
  "cwd": "/path/to/project",
  "messages": [
  ],
  "role": "user",
  "content": "Hello",
  "timestamp":
    "2024-01-01T00:00:00Z"
  },
}
```

20

Branching

- Sessions support in-place branching:
- Use /tree to view session history
- Select any point to branch from
- New branches don't create new files
- Switch between branches seamlessly

Context Compaction

- Automatic compaction when context window is full
- Manual compaction via /compact command
- Configurable compaction strategy

21